Sheetalshee Update README.md		c8e16e2 · now
.gitignore	Initial commit	2 years ago
LICENSE	Initial commit	2 years ago
README.md	Update README.md	now

X8 Web Scraping On E-commerce platform using BeautifulSoup

AIM: To perform Web Scraping on Amazon using (beautifulsoup) Python.

Description:

Web scraping is the process of extracting data from various websites and parsing it. In other words, it's a technique to extract unstructured data and store that data either in a local file or in a database. There are many ways to collect data that involve a huge amount of hard work and consume a lot of time. Web scraping can save programmers many hours. Beautiful Soup is a Python web scraping library that allows us to parse and scrape HTML and XML pages. One can search, navigate, and modify data using a parser. It's versatile and saves a lot of time.
The basic steps involved in web scraping are:

1) Loading the document (HTML content)

2) Parsing the document

3) Extraction

4) Transformation

Procedure:

1. Import necessary libraries (requests, BeautifulSoup, re, matplotlib.pyplot).
2. Define convert_price_to_float(price) Function: to Remove non-numeric characters from a price string and convert it to a float.
3. Define get_amazon_products(search_query) Function: to Scrape Amazon for product information based on the search query.
4. Fetch and parse the HTML content then Extract product names and prices from the search results and Sort product information based on converted prices in ascending order.
5. Return sorted product data as a list of dictionaries.
6. Call get_amazon_products(search_query) to get product data based on the user's search query.
7. Check if products are found; if not, display "No products found."
8. Visualize Product Data using a Bar Chart

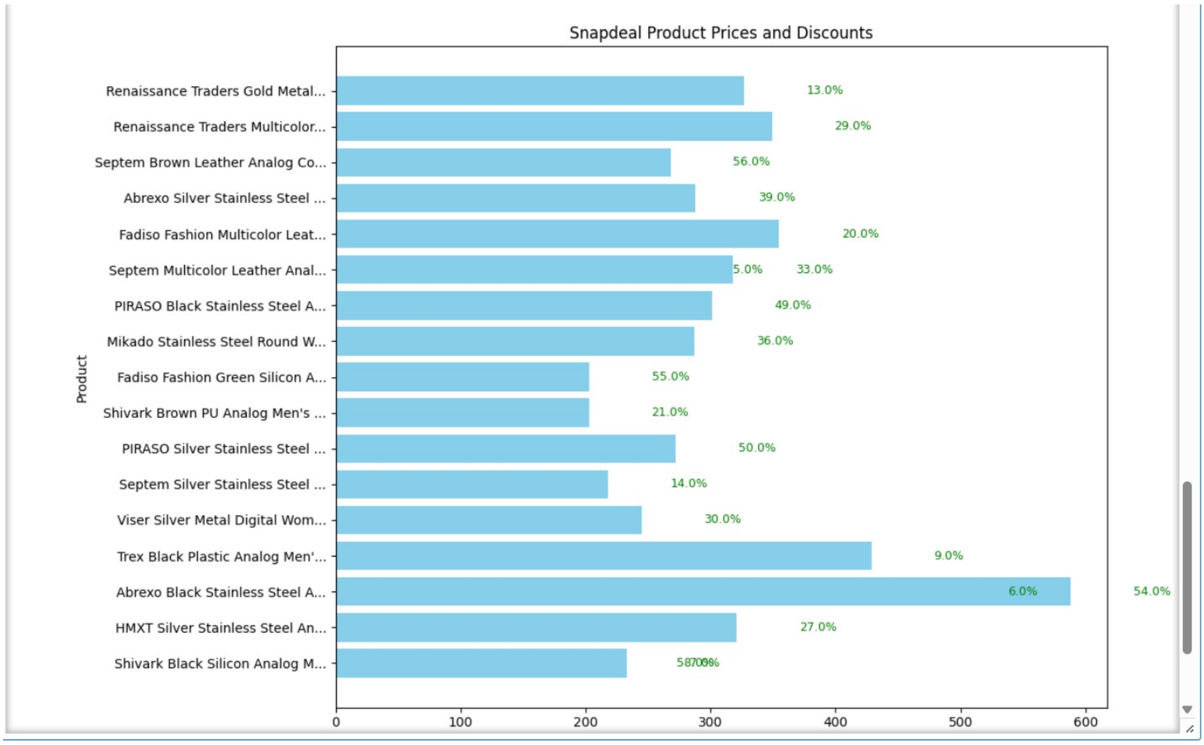
Program:

```
import requests
from bs4 import BeautifulSoup
import matplotlib.pyplot as plt
import re
import random

def convert_price_to_float(price_str):
    clean_price = re.sub(r'[^\d.]', '', price_str)
    return float(clean_price) if clean_price else 0.0

def get_snapdeal_products(search_query):
    url = f'https://www.snapdeal.com/search?keyword={search_query.replace("
    headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKi
    }

    response = requests.get(url, headers=headers)
    products_data = []

    if response.status_code == 200:
        soup = BeautifulSoup(response.content, 'html.parser')
```

```
        products = soup.find_all('div', {'class': 'product-tuple-listing'})

        for product in products:
            title = product.find('p', {'class': 'product-title'})
            price = product.find('span', {'class': 'product-price'})
            original_price = product.find('span', {'class': 'product-desc-p
            rating = product.find('div', {'class': 'filled-stars'})

            if title and price:
                product_name = title.text.strip()
                product_price = convert_price_to_float(price.get('data-pric
                product_original_price = convert_price_to_float(original_pr

                if product_original_price > product_price:
                    discount_percent = round(((product_original_price - pro
                else:
                    discount_percent = random.randint(5, 70)  # assign rand

                product_rating = rating['style'].split(':')[-1].replace('%'

                products_data.append({
                    'Product': product_name,
                    'Price (₹)': product_price,
                    'Original Price (₹)': product_original_price,
                    'Discount (%)': discount_percent,
                    'Rating': product_rating
                })

                print(f'Product: {product_name}')
                print(f'Price: ₹{product_price}')
                print(f'Original Price: ₹{product_original_price}')
                print(f'Discount: {discount_percent}%')
                print(f'Rating: {product_rating}')
                print('---')

    else:
        print('Failed to retrieve content from Snapdeal')

    return products_data

def visualize_product_data(products):
    if products:
        product_names = [p['Product'][:30] + '...' if len(p['Product']) > 3
        prices = [p['Price (₹)'] for p in products]
        discounts = [p['Discount (%)'] for p in products]

        plt.figure(figsize=(12, 8))
        bars = plt.barh(product_names, prices, color='skyblue')

        for i, (bar, discount) in enumerate(zip(bars, discounts)):
            plt.text(bar.get_width() + 50, bar.get_y() + bar.get_height()/2
```

```
                     f'{discount:.1f}%', va='center', fontsize=9, color='gr

        plt.xlabel('Price (₹)')
        plt.ylabel('Product')
        plt.title('Snapdeal Product Prices and Discounts')
        plt.tight_layout()
        plt.show()
    else:
        print('No products to display.')

search_query = input('Enter product to search on Snapdeal: ')
products = get_snapdeal_products(search_query)
visualize_product_data(products)
```

Output:



WITH DISCOUNT PERCENTAGE:

```
Enter product to search on Snapdeal:  watch
Product: Shivark Black Silicon Analog Men's Watch
Price: ₹223.0
Original Price: ₹223.0
Discount: 58%
Rating: 84.00000000000001
---
Product: HMXT Silver Stainless Steel Analog Men's Watch
Price: ₹321.0
Original Price: ₹321.0
Discount: 27%
Rating: 84.00000000000001
---
Product: Abrexo Black Stainless Steel Analog Men's Watch
Price: ₹488.0
Original Price: ₹488.0
Discount: 6%
Rating: 74.0
---
Product: Trex Black Plastic Analog Men's Watch
Price: ₹429.0
Original Price: ₹429.0
Discount: 9%
```

```
Rating: 90.0
---
Product: Abrexo Black Stainless Steel Analog Men's Watch
Price: ₹588.0
Original Price: ₹588.0
Discount: 54%
Rating: 72.0
---
Product: Viser Silver Metal Digital Womens Watch
Price: ₹245.0
Original Price: ₹245.0
Discount: 30%
Rating: 72.0
---
```

Result:

Verified successfully

Releases

No releases published

Packages

No packages published

Contributors

No contributors